

8.3 LoRA, Adapters, and Federated Updates

Train BA , freeze W . $r(d+k)$ trainable, not dk . Merge has no extra decode latency.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Full fine-tuning moves every entry of W . **LoRA** (Hu et al., 2021) freezes W and learns a **low-rank residual**. You store a few megabytes per task instead of a second copy of the 7B.

1. What LoRA is

Full fine-tuning copies every weight and walks off the old task (note **8.2**). **LoRA** freezes W and trains a low-rank residual BA . For a linear map $W \in \mathbb{R}^{d \times k}$,

$$h = Wx + \frac{\alpha}{r}BAx,$$

with $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$, and $\text{rank } r \ll \min(d, k)$. Initialize $B = 0$ so the adapter starts as the base model. Train only A, B (and maybe a bias). The reason for LoRA is that a full Δ has the same size as W (GPT-3: 175B extra weights per task). Encode $\Delta W = BA$ instead.

You are not inventing a new transformer. You are storing a few megabytes per skill instead of a second 7B. One adapter per skill is how you avoid the XOR-then-AND overwrite: the base stays, the skill is a file. QLoRA is the same algebra with W stored in 4-bit and the adapters in float.

2. Why we use it

A full fine-tune is expensive to store and destructive to run. GPT-3-scale Δ is 175B extra numbers per task. Even a 7B copy per hospital is the wrong footprint if the base can stay frozen. LoRA is the cheap “add B without moving A” answer that note **8.2** promised.

Small r (4, 8, 16) is often enough for style and instruction shifts; harder domain shifts may want larger r or more layers. At deploy time you can **merge** $W \leftarrow W + (\alpha / r)BA$ and throw the adapter away, or keep several adapters and swap them. Merge means **no extra inference latency**. Swap means one base, many skills.

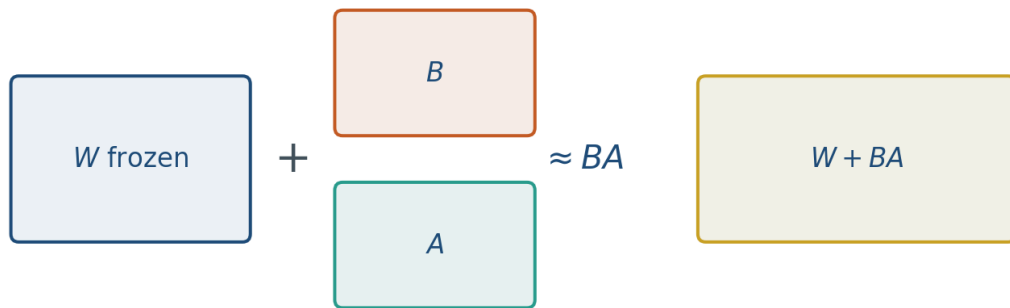
For a project: report r , which modules you adapted, α , and whether you merged. Compare to a small full fine-tune if you can afford it. Lab 8 uses a 4×4 frozen W so you can print BA .

3. Architecture

Insert A, B on attention projections and/or MLP up/down maps. The frozen W still computes Wx . The adapter adds $(\alpha / r)BAx$. Init $B = 0$ so step 0 is the base. Attention projections (q, k, v, o) are the usual

insertion points; MLP maps are allowed too.

LoRA: train a thin pair BA , leave W alone



CS224N L11: a full copy of W is 175B extra for GPT-3. LoRA is this table.

$d = k = 4096, r = 8$	count
dk	$4096^2 = 16.8\text{M}$
$r(d + k)$	$8 \cdot 8192 = 65,536$
ratio	0.39%

Trainable count for one matrix is $r(d + k)$, not dk , and not r^2 . QLoRA quantizes W to 4-bit and still trains float adapters. Same picture: the big matrix is frozen (and cheap to store); the small factors move.

Why not init $A = 0$ instead? Either factor zero works for a zero start; the usual recipe is Gaussian A and **zero** B so the first step can still move (nonzero A, B getting a gradient). If both start at 0, the product stays 0 until you break the deadlock (you would not).

4. How it works, step by step

1. **Freeze** W . Insert A (Gaussian) and $B = 0$ on the chosen projections.
2. **Train** only A, B on the new task (SFT, or later a preference loss). The base does not walk off task A .
3. **Scale** with α / r . That ratio does not add parameters; it is a knob on adapter strength.
4. **Deploy**. Merge $W \leftarrow W + (\alpha / r)BA$ for a single file with base latency, or keep A, B on disk and swap adapters by subtracting one BA and adding another. Merge loses easy unmerge unless you kept A, B .

5. **Report** r , modules, α , merged or not. Small r may fail a hard domain shift; raising r or adapting more layers is the next knob, not “LoRA does not work.”

Lab size: $r = 1$, $d = k = 4$. $A \in \mathbb{R}^{1 \times 4}$ (4 numbers), $B \in \mathbb{R}^{4 \times 1}$ (4 numbers), **8** trainable vs **16** in W . At init $B = 0 \Rightarrow BA = 0 \Rightarrow h = Wx$. That printout is Lab 8 section 1.

5. Mathematical formulas

Forward with a scaled low-rank residual:

$$h = Wx + \frac{\alpha}{r}BAx.$$

Trainable count per adapted matrix:

$$r(d + k),$$

versus dk in a full W . One map, $d = k = 4096$, rank $r = 8$:

$$r(d + k) = 8 \cdot (4096 + 4096) = 65,536$$

trainable numbers. Full W has $4096^2 = 16,777,216$ entries. Ratio $65536 / 16777216 \approx 0.0039$ (about **0.39%**).

Merge at deploy:

$$W' = W + \frac{\alpha}{r}BA,$$

which is $d \times k$ again. Inference cost matches the base.

6. Positive points and negative points

Positive.

- Cheap: $r(d + k)$ trainable numbers per matrix instead of dk , megabytes per task instead of a second 7B.
- Mergeable: after $W \leftarrow W + (\alpha / r)BA$ there is no extra inference latency.
- One adapter per skill keeps the base frozen, which is the forgetting fix replay cannot promise.
- QLoRA keeps the same algebra with 4-bit W .

Negative.

- Small r may fail hard domain shifts; you may need a larger rank or more layers.
- Init both factors randomly and the model is not the base at step 0.
- Counting LoRA as r^2 or as $2r$ only is a report bug; it is $r(d + k)$ per adapted matrix.

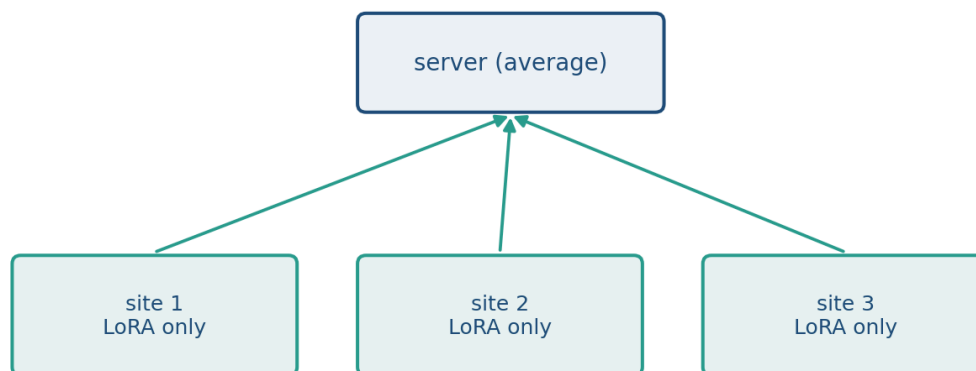
- Merge throws away easy unmerge unless you kept A, B .
- Not a new architecture. A project that “used LoRA” still has to name r , modules, and α .

When not to. If you must change every feature and you can afford a full copy, full fine-tune is simpler. If the model already fits and you only needed fewer bits, that is quantization (note **7.3**), not an adapter.

7. Federated adapters

Sites that cannot ship raw text can still train a LoRA locally and send A, B to a server. The server averages adapters (or stacks them) without seeing documents. That is **federated** fine-tuning in the sense this course needs: communication is the adapter, not the corpus.

Federated sketch: ship adapters, not raw documents



Privacy is not automatic. Adapters can leak. A hospital that trains a LoRA and sends it to you did **not** send the notes, but the adapter can still carry traces of them. Shipping a hospital adapter and calling it private is the usual overclaim. The footprint is the right size; the privacy theorem is not included.

8. Teaching this note

About **35–40 minutes** at the board: write $r(d + k)$ vs dk , init $B = 0$, merge vs swap. Play the **full** LoRA video as assigned (in class, first 20–25 min if the clip runs long; rest after). Lab 8 section 1 is the 4×4 printout.

9. Worked example

One map, $d = k = 4096$, rank $r = 8$:

$$r(d + k) = 8 \cdot (4096 + 4096) = 65,536$$

trainable numbers. Full W has $4096^2 = 16,777,216$ entries. Ratio $65536 / 16777216 \approx 0.0039$ (about **0.39%**). α / r is a scale; it does not add parameters.

Lab size: $r = 1$, $d = k = 4$. $A \in \mathbb{R}^{1 \times 4}$ (4 numbers), $B \in \mathbb{R}^{4 \times 1}$ (4 numbers), **8** trainable vs **16** in W . At init $B = 0 \Rightarrow BA = 0 \Rightarrow h = Wx$.

Why not init $A = 0$ instead? Either factor zero works for a zero start; the usual recipe is Gaussian A and **zero** B so the first step can still move (nonzero A , B getting a gradient). If both start at 0, the product stays 0 until you break the deadlock (you would not).

Merge: $W' = W + (\alpha / r)BA$ is $d \times k$ again. Inference cost matches the base; you lose easy unmerge unless you kept A, B .

10. Where students get stuck

- Counting LoRA as r^2 or as $2r$ only. It is $r(d + k)$ per adapted matrix.
 - Init both factors randomly so the model is not the base at step 0.
 - Shipping a hospital adapter and calling it private. The corpus stayed, but adapters can still leak.
-

11. Video

[Umar Jamil — LoRA, explained visually + PyTorch from scratch](#). Play the **full** video as assigned. In a two-hour class, run **0:00–~25:00** (math + the BA diagram) and leave the PyTorch walkthrough for after, or play through if time. Pause when B is zeros: that is Lab 8's first print.

12. Practice

1. If $r = 1$ and $d = k = 4$, how many trainable numbers are in (A, B) versus W ?
2. Why initialize $B = 0$ rather than $A = 0$?
3. A hospital trains a LoRA and sends it to you. What did they *not* send, and what can still leak?
4. $d = 1024$, $k = 4096$, $r = 16$. Compute $r(d + k)$ and the ratio to dk .
5. You adapt q, k, v, o projections, each $d \times d$ with $d = 2048$, $r = 8$. Ignore α . How many trainable parameters is that (four matrices)?